

Tarea Semana 4: Simulador de Lotería Nacional

Librerías y Módulos en Python

Curso de Programación en Python

Tiempo de entrega: 1 semana

1. Introducción

¡Bienvenido a la Semana 4! Has aprendido los fundamentos de Python: variables, condicionales, bucles y manejo de errores. Ahora es momento de ampliar tu poder de programación usando **librerías y módulos**.

Las librerías son colecciones de código pre-escrito que puedes usar en tus programas. ¿Por qué reinventar la rueda cuando alguien ya creó una solución perfecta? Python tiene una vasta biblioteca estándar y miles de paquetes externos disponibles.

En esta tarea, crearás un simulador completo de la Lotería Nacional de Costa Rica, usando múltiples librerías para generar números aleatorios, calcular estadísticas, trabajar con fechas, interactuar con el sistema operativo, y más.

2. Objetivos de Aprendizaje

Al completar esta tarea, serás capaz de:

- Importar módulos con `import`
- Usar importación selectiva con `from ... import ...`
- Trabajar con módulos de la biblioteca estándar de Python
- Usar `random` para generación de números aleatorios
- Usar `statistics` para cálculos estadísticos
- Usar `datetime` para manejo de fechas y horas
- Usar `os` y `sys` para interacción con el sistema
- Instalar paquetes externos con `pip`
- Crear tus propios módulos personalizados
- Organizar código en múltiples archivos
- Entender la diferencia entre módulos y paquetes

3. Enunciado del Proyecto

3.1. Descripción General

Vas a crear un programa llamado “**Simulador de Lotería Nacional de Costa Rica**”. Este programa simula el sorteo de la Lotería Nacional, permite a usuarios comprar billetes virtuales, realizar sorteos, calcular premios, mantener estadísticas, y guardar/cargar datos.

3.2. Contexto de la Lotería Nacional

La Lotería Nacional de Costa Rica funciona así:

- Los billetes tienen números de 00000 a 99999 (5 dígitos)
- Cada billete tiene una serie (A, B, C, etc.)
- El premio mayor es para el número exacto
- Hay premios para aproximaciones (un número antes/después)
- Hay premios para terminaciones (últimos 2, 3, 4 dígitos)
- Los sorteos se realizan en fechas específicas

4. Requisitos Funcionales

4.1. Parte 1: Módulos a Utilizar

Tu programa DEBE usar las siguientes librerías:

4.1.1. 1. Módulo random

Listing 1: Uso de random para sorteos

```
1 import random
2
3 # Generar numero ganador (00000-99999)
4 numero_ganador = random.randint(0, 99999)
5
6 # Generar serie ganadora (A-Z)
7 series = ['A', 'B', 'C', 'D', 'E']
8 serie_ganadora = random.choice(series)
9
10 # Simular venta de billetes aleatorios
11 billetes_vendidos = random.sample(range(100000), 1000)
12
13 # Generar multiples ganadores para aproximaciones
14 aproximaciones = [numero_ganador - 1, numero_ganador + 1]
```

4.1.2. 2. Módulo statistics

Listing 2: Uso de statistics para analisis

```
1 import statistics
2
3 # Lista de precios de billetes vendidos
4 precios = [10000, 10000, 10000, 5000, 5000]
5
6 # Calcular estadísticas de ventas
7 promedio_precio = statistics.mean(precios)
8 mediana_precio = statistics.median(precios)
9 moda_precio = statistics.mode(precios)
10
11 # Calcular desviación estandar de numeros jugados
12 numeros_jugados = [12345, 54321, 98765, 11111]
13 desv_std = statistics.stdev(numeros_jugados)
```

4.1.3. 3. Módulo datetime

Listing 3: Uso de datetime para fechas

```
1 from datetime import datetime, timedelta
2
3 # Obtener fecha y hora actual
4 fecha_actual = datetime.now()
5 print(f"Sorteo realizado: {fecha_actual.strftime('%d/%m/%Y %H:%M:%S')}")
6
7 # Calcular proxima fecha de sorteo (cada domingo)
8 dias_hasta_domingo = (6 - fecha_actual.weekday()) % 7
9 proxima_fecha = fecha_actual + timedelta(days=dias_hasta_domingo)
10
11 # Registrar tiempo de sorteo
12 inicio = datetime.now()
13 # ... realizar sorteo ...
14 fin = datetime.now()
15 duracion = (fin - inicio).total_seconds()
16 print(f"Sorteo completo en {duracion} segundos")
```

4.1.4. 4. Módulo os

Listing 4: Uso de os para sistema de archivos

```
1 import os
2
3 # Verificar si existe carpeta de datos
4 if not os.path.exists('datos_loteria'):
5     os.makedirs('datos_loteria')
6     print("Carpeta de datos creada")
7
8 # Listar archivos de sorteos anteriores
9 archivos = os.listdir('datos_loteria')
```

```
10 print(f"Sorteos guardados: {len(archivos)}")
11
12 # Obtener informacion del sistema
13 print(f"Sistema operativo: {os.name}")
14 print(f"Directorio actual: {os.getcwd()}")
```

4.1.5. 5. Módulo sys

Listing 5: Uso de sys para informacion del sistema

```
1 import sys
2
3 # Mostrar version de Python
4 print(f"Python version: {sys.version}")
5
6 # Verificar argumentos de linea de comandos
7 if len(sys.argv) > 1:
8     modo = sys.argv[1]
9     print(f"Modo: {modo}")
10
11 # Salir del programa con codigo de estado
12 if error_critico:
13     print("Error critico detectado")
14     sys.exit(1)
```

4.1.6. 6. Librería externa: colorama (opcional)

Listing 6: Instalacion y uso de colorama para colores

```
1 # Instalar con: pip install colorama
2
3 from colorama import Fore, Back, Style, init
4
5 # Inicializar colorama
6 init(autoreset=True)
7
8 # Imprimir mensajes con colores
9 print(Fore.GREEN + "FELICIDADES! HAS GANADO!")
10 print(Fore.RED + "Lo sentimos, no ganaste esta vez")
11 print(Fore.YELLOW + "Numero ganador: " + Fore.CYAN + "12345")
12 print(Back.WHITE + Fore.BLACK + "Premio Mayor")
```

4.2. Parte 2: Crear Módulo Propio

Debes crear al menos UN módulo personalizado llamado `utilidades_loteria.py`:

Listing 7: Archivo `utilidades_loteria.py`

```
1 """
2 Modulo de utilidades para el simulador de loteria
3 Contiene funciones auxiliares reutilizables
4 """
5
```

```

6 def formatear_numero(numero):
7     """Formatea un numero de loteria con ceros a la izquierda"""
8     return str(numero).zfill(5)
9
10 def validar_numero_loteria(numero):
11     """Valida que un numero este en rango valido (0-99999)"""
12     try:
13         num = int(numero)
14         return 0 <= num <= 99999
15     except ValueError:
16         return False
17
18 def calcular_premio(numero_jugado, numero_ganador, tipo="exacto"):
19     """Calcula el premio segun tipo de acierto"""
20     premios = {
21         "exacto": 200000000, # 200 millones
22         "aproximacion": 50000000, # 50 millones
23         "3_cifras": 500000, # 500 mil
24         "2_cifras": 100000 # 100 mil
25     }
26     return premios.get(tipo, 0)
27
28 def obtener_terminaciones(numero):
29     """Obtiene las terminaciones de 2, 3 y 4 cifras"""
30     num_str = str(numero).zfill(5)
31     return {
32         "2_cifras": num_str[-2:],
33         "3_cifras": num_str[-3:],
34         "4_cifras": num_str[-4:]
35     }

```

Luego importar y usar en tu programa principal:

Listing 8: Importar modulo propio en main.py

```

1 # Importar modulo completo
2 import utilidades_loteria
3
4 numero = utilidades_loteria.formatear_numero(1234)
5
6 # O importar funciones especificas
7 from utilidades_loteria import validar_numero_loteria, calcular_premio
8
9 if validar_numero_loteria(numero_jugado):
10     premio = calcular_premio(numero_jugado, ganador, "exacto")

```

4.3. Parte 3: Funcionalidades del Programa

4.3.1. Menú Principal

1. Comprar billetes de lotería
2. Realizar sorteo
3. Ver mis billetes

4. Ver resultados del último sorteo
5. Ver estadísticas generales
6. Verificar si gané
7. Información del sistema
8. Salir

4.3.2. Comprar Billetes

- Generar número aleatorio o elegir número específico
- Elegir serie (A-E) o aleatoria
- Cada billete cuesta C\$10,000
- Permitir comprar múltiples billetes
- Usar `random.randint()` para números aleatorios
- Guardar billetes del usuario en una lista
- Mostrar resumen de compra con fecha (`datetime`)

4.3.3. Realizar Sorteo

- Generar número ganador (00000-99999) con `random`
- Generar serie ganadora con `random.choice()`
- Generar aproximaciones (número -1 y +1)
- Registrar fecha y hora del sorteo con `datetime`
- Calcular todos los premios:
 - Premio mayor (número exacto + serie)
 - Aproximaciones
 - 4 cifras (últimos 4 dígitos)
 - 3 cifras (últimos 3 dígitos)
 - 2 cifras (últimos 2 dígitos)
- Guardar resultados del sorteo

4.3.4. Verificar Premios

- Comparar billetes del usuario con resultados
- Usar módulo propio para calcular premios
- Mostrar si ganó y cuánto
- Actualizar estadísticas de ganancias

4.3.5. Estadísticas

Usar `statistics` para calcular:

- Total de billetes comprados
- Total invertido
- Total ganado
- Promedio de números jugados
- Número más jugado (moda)
- Serie más jugada
- Rentabilidad (ganado/invertido)
- Historial de sorteos realizados

4.3.6. Información del Sistema

Usar `os` y `sys` para mostrar:

- Versión de Python
- Sistema operativo
- Directorio actual
- Archivos guardados
- Espacio usado por datos

4.4. Ejemplo de Ejecución

Listing 9: Flujo completo del programa

```
1  =====
2  SIMULADOR LOTERIA NACIONAL CR
3  =====
4  Python 3.11.5 | Sistema: Windows
5  Fecha: 24/10/2025 14:30:15
6
7  --- MENU PRINCIPAL ---
8  1. Comprar billetes
9  2. Realizar sorteo
10 3. Ver mis billetes
11 4. Resultados ultimo sorteo
12 5. Estadisticas
13 6. Verificar premios
14 7. Info del sistema
15 8. Salir
16
17 Opcion: 1
18
19 --- COMPRAR BILLETES ---
20 1. Numero aleatorio
21 2. Elegir numero especifico
22
23 Opcion: 1
24
25 Generando numero aleatorio...
26 Numero: 45678
27 Serie: C
28 Precio: 10 ,000
29
30 Confirmar compra? (s/n): s
31
32 Billeto comprado exitosamente!
33 Fecha de compra: 24/10/2025 14:31:00
34
35 Comprar otro? (s/n): s
36
37 Opcion: 2
38 Ingresa el numero (00000-99999): 12345
39 Serie (A-E) o random (r): A
40 Precio: 10 ,000
41
42 Billeto comprado exitosamente!
43
44 Comprar otro? (s/n): n
45
46 Total invertido: 20 ,000
47 Billetoes comprados: 2
48
49 --- MENU PRINCIPAL ---
50 Opcion: 3
```



```
51
52 --- MIS BILLETES ---
53 1. Numero: 45678 | Serie: C | Comprado: 24/10/2025 14:31
54 2. Numero: 12345 | Serie: A | Comprado: 24/10/2025 14:32
55
56 Total: 2 billetes
57
58 --- MENU PRINCIPAL ---
59 Opcion: 2
60
61 --- REALIZANDO SORTEO ---
62 Fecha del sorteo: 24/10/2025 14:35:00
63
64 Generando numero ganador...
65 Mezclando bolas...
66 Seleccionando serie...
67
68 NUMERO GANADOR: 12346
69 SERIE GANADORA: A
70
71 Aproximaciones:
72 - 12345 (aproximacion anterior)
73 - 12347 (aproximacion posterior)
74
75 Premios por terminaciones:
76 - 4 cifras (2346): 2 ,000,000
77 - 3 cifras (346): 500 ,000
78 - 2 cifras (46): 100 ,000
79
80 Sorteo completado en 0.045 segundos
81
82 --- MENU PRINCIPAL ---
83 Opcion: 6
84
85 --- VERIFICAR PREMIOS ---
86 Revisando tus 2 billetes...
87
88 Billeto #1: 45678 - Serie C
89 Terminacion 2 cifras: NO
90 Terminacion 3 cifras: NO
91 Sin premio
92
93 Billeto #2: 12345 - Serie A
94 APROXIMACION ANTERIOR!
95 Premio: 50 ,000,000
96
97 FELICIDADES! HAS GANADO 50 ,000,000!
98
99 Tu inversion: 20 ,000
100 Tus ganancias: 50 ,000,000
101 Rentabilidad: 250,000%
102
103 --- MENU PRINCIPAL ---
```

```
104 Opcion: 5
105
106 --- ESTADISTICAS GENERALES ---
107
108 Datos de Usuario:
109 - Total billetes comprados: 2
110 - Total invertido: 20 ,000
111 - Total ganado: 50 ,000,000
112 - Rentabilidad: 250,000%
113
114 Analisis de Numeros Jugados:
115 - Numeros: [45678, 12345]
116 - Promedio: 29,011.5
117 - Numero mas jugado: N/A (todos diferentes)
118
119 Series Jugadas:
120 - Serie A: 1 vez
121 - Serie C: 1 vez
122
123 Historial de Sorteos:
124 - Sorteos realizados: 1
125 - Ultimo sorteo: 24/10/2025 14:35
126 - Numero ganador: 12346 - Serie A
127
128 --- MENU PRINCIPAL ---
129 Opcion: 7
130
131 --- INFORMACION DEL SISTEMA ---
132
133 Python:
134 - Version: 3.11.5
135 - Path: C:\Python311\python.exe
136
137 Sistema Operativo:
138 - OS: Windows (nt)
139 - Directorio actual: C:\Users\user\loteria
140
141 Archivos:
142 - Sorteos guardados: 1
143 - Billetes guardados: 2
144 - Carpeta datos: datos_loteria/
145
146 Modulos cargados:
147 - random
148 - statistics
149 - datetime
150 - os
151 - sys
152 - utilidades_loteria
153
154 --- MENU PRINCIPAL ---
155 Opcion: 8
156
```

```
157 Guardando datos...
158 Gracias por usar el Simulador de Loteria!
159
160 Hasta luego!
```

5. Requisitos Técnicos Detallados

5.1. Importaciones Requeridas

Tu archivo principal debe incluir:

Listing 10: Importaciones del programa principal

```

1  # Modulos de biblioteca estandar
2  import random
3  import statistics
4  import os
5  import sys
6  from datetime import datetime, timedelta
7
8  # Modulo propio
9  import utilidades_loteria
10 # 0 importacion selectiva:
11 # from utilidades_loteria import formatear_numero, calcular_premio
12
13 # Libreria externa (opcional pero recomendado)
14 try:
15     from colorama import Fore, Style, init
16     init(autoreset=True)
17     COLORAMA_DISPONIBLE = True
18 except ImportError:
19     print("Colorama no instalado. Ejecutar: pip install colorama")
20     COLORAMA_DISPONIBLE = False

```

5.2. Estructura de Archivos

Tu proyecto debe tener esta estructura:

Listing 11: Estructura del proyecto

```

1  loteria/
2      main.py                # Programa principal
3      utilidades_loteria.py  # Modulo personalizado
4      datos_loteria/        # Carpeta para datos (se crea
5          automaticamente)
6          sorteos.txt
7          billetes.txt
8          estadisticas.txt
9      requirements.txt       # Lista de dependencias
10     README.txt            # Instrucciones

```

5.3. Archivo requirements.txt

Crear archivo con dependencias:

Listing 12: requirements.txt

```

1  colorama >=0.4.6

```

Instrucciones para instalar:

```
1 pip install -r requirements.txt
```

5.4. Uso Correcto de Módulos

5.4.1. Random

Debes usar al menos 3 funciones diferentes:

```
1 import random
2
3 # randint: numero entero aleatorio en rango
4 numero = random.randint(0, 99999)
5
6 # choice: elegir elemento aleatorio de lista
7 serie = random.choice(['A', 'B', 'C', 'D', 'E'])
8
9 # sample: multiples elementos sin repeticion
10 numeros_vendidos = random.sample(range(100000), 50)
11
12 # shuffle: mezclar lista (in-place)
13 lista = [1, 2, 3, 4, 5]
14 random.shuffle(lista)
15
16 # random: float entre 0 y 1
17 probabilidad = random.random()
18 if probabilidad < 0.01: # 1% de probabilidad
19     print("Bonus especial!")
```

5.4.2. Statistics

Debes usar al menos 3 funciones:

```
1 import statistics
2
3 numeros = [12345, 54321, 11111, 99999, 12345]
4
5 # mean: promedio
6 promedio = statistics.mean(numeros)
7
8 # median: mediana
9 mediana = statistics.median(numeros)
10
11 # mode: moda (mas frecuente)
12 moda = statistics.mode(numeros)
13
14 # stdev: desviacion estandar
15 desviacion = statistics.stdev(numeros)
16
17 # variance: varianza
18 varianza = statistics.variance(numeros)
```

5.4.3. Datetime

Debes usar funcionalidades de fecha/hora:

```
1 from datetime import datetime, timedelta
2
3 # Obtener fecha/hora actual
4 ahora = datetime.now()
5
6 # Formatear fecha
7 fecha_str = ahora.strftime("%d/%m/%Y %H:%M:%S")
8
9 # Parsear fecha desde string
10 fecha = datetime.strptime("24/10/2025", "%d/%m/%Y")
11
12 # Calcular diferencias de tiempo
13 proximo_sorteo = ahora + timedelta(days=7)
14 diferencia = proximo_sorteo - ahora
15 dias_restantes = diferencia.days
16
17 # Obtener componentes
18 anio = ahora.year
19 mes = ahora.month
20 dia = ahora.day
21 hora = ahora.hour
```

5.5. Validaciones con Try-Except

Todas las importaciones y operaciones deben manejarse:

```
1 # Manejar modulos opcionales
2 try:
3     from colorama import Fore
4     TIENE_COLOR = True
5 except ImportError:
6     TIENE_COLOR = False
7     print("Instalando sin colores...")
8
9 # Manejar operaciones de archivo
10 try:
11     with open('datos.txt', 'r') as f:
12         datos = f.read()
13 except FileNotFoundError:
14     print("Archivo no encontrado, creando nuevo...")
15     datos = ""
16 except PermissionError:
17     print("Sin permisos para leer archivo")
```

6. Consejos y Recomendaciones

- **Lee la documentación:** Cada módulo tiene documentación oficial. Úsala.
- **Importa solo lo necesario:** No hagas `from modulo import *`

- **Organiza las importaciones:**

1. Biblioteca estándar primero
2. Librerías externas después
3. Módulos propios al final

- **Usa alias cuando sea útil:** `import statistics as stats`

- **Comenta módulos propios:** Usa docstrings en funciones

- **Prueba instalación de paquetes:** Verifica que `pip` funciona

- **Maneja ImportError:** Los módulos externos pueden no estar instalados

- **No reinventes la rueda:** Si existe un módulo que hace lo que necesitas, úsalo

7. Entregables

1. `main.py` - Programa principal
2. `utilidades_loteria.py` - Módulo personalizado
3. `requirements.txt` - Dependencias
4. `README.txt` - Instrucciones de instalación y uso
5. Carpeta `datos_loteria/` se crea automáticamente

8. Defensa del Código (60 % de la nota final)

8.1. ¿Qué es la defensa del código?

La defensa del código es una reunión virtual donde presentarás y explicarás tu trabajo. Esta defensa es **obligatoria** y representa el **60 % de la nota final** de esta tarea.

8.2. ¿Cómo funciona?

Durante la defensa deberás:

1. **Compartir tu pantalla** mostrando tu código y estructura de archivos.
2. **Demostrar instalación de dependencias:**
 - Mostrar archivo `requirements.txt`
 - Ejecutar `pip install -r requirements.txt`
 - Verificar que `colorama` se instala correctamente
3. **Ejecutar el programa completo:**
 - Comprar billetes
 - Realizar sorteo
 - Verificar premios

- Mostrar estadísticas
- Ver información del sistema

4. Explicar el uso de cada módulo:

- Por qué usaste `random` y qué funciones
- Cómo usaste `statistics` para cálculos
- Para qué usaste `datetime`
- Qué haces con `os` y `sys`
- Cómo creaste y usas tu módulo personalizado

5. Responder preguntas como:

- “¿Cuál es la diferencia entre `import random` y `from random import randint`?”
- “¿Qué hace `random.seed()`? ¿Para qué serviría?”
- “¿Cómo instalaste `colorama`?”
- “¿Qué pasa si `colorama` no está instalado?”
- “¿Por qué crear un módulo propio?”
- “¿Cómo funciona la importación de módulos en Python?”
- “¿Qué es `requirements.txt`?”

6. Demostrar comprensión de:

- Diferencia entre módulos de biblioteca estándar y externos
- Cómo instalar paquetes con `pip`
- Cómo crear y estructurar módulos propios
- Buenas prácticas de importación

8.3. Preguntas Típicas de la Defensa

Prepárate para responder:

- ¿Cuál es la diferencia entre `import random` y `from random import *`?
- ¿Qué es `pip` y para qué sirve?
- ¿Cómo verificas si un módulo está instalado?
- ¿Qué hace `if __name__ == "__main__":`?
- ¿Por qué usar `try-except` al importar `colorama`?
- ¿Qué es un docstring y por qué usarlo?
- ¿Cómo se organiza un proyecto con múltiples archivos?
- Dame 3 funciones de `random` y para qué sirven
- ¿Qué diferencia hay entre `mean` y `median`?

8.4. Consejos para una Buena Defensa

- **Conoce cada módulo:** Lee la documentación de los que usaste
- **Instala antes:** Verifica que pip funciona y colorama se instala
- **Explica tu módulo:** Entiende cada función que creaste
- **Ten ejemplos:** Muestra diferentes funcionalidades en acción
- **Entiende imports:** Sé claro sobre las diferencias de importación

8.5. Distribución de la Nota

- **Código funcional (40 %):** El programa usa correctamente múltiples librerías
- **Defensa del código (60 %):** Tu capacidad de explicar módulos y librerías

Importante: Si no asistes a la defensa o no puedes explicar las librerías que usaste, la nota máxima será 40/100.

9. Desafíos Opcionales (Puntos Extra)

1. Módulo JSON para persistencia (+6 pts):

- Usar `json` para guardar/cargar datos
- Guardar historial de sorteos en formato JSON
- Cargar datos anteriores al iniciar

2. Gráficos con `matplotlib` (+8 pts):

- Instalar: `pip install matplotlib`
- Generar gráfico de barras de números más jugados
- Gráfico de pastel de distribución de premios
- Histograma de números ganadores históricos

3. Módulo `collections` (+5 pts):

- Usar `Counter` para contar frecuencias
- Usar `defaultdict` para organizar datos
- Mostrar top 10 números más jugados

4. Módulo `argparse` (+5 pts):

- Agregar argumentos de línea de comandos
- `python main.py --sorteo` para hacer sorteo directo
- `python main.py --stats` para ver estadísticas

5. Módulo `pickle` para datos binarios (+4 pts):

- Guardar objetos complejos con `pickle`
- Serializar y deserializar datos

- Comparar con JSON

6. Crear paquete completo (+7 pts):

- Organizar en múltiples módulos
- Crear `__init__.py` para paquete
- Estructura: `loteria/core/`, `loteria/utils/`, `loteria/data/`

Nota: Los puntos extra solo se otorgan si la implementación es correcta Y puedes explicarla en la defensa.

10. Rúbrica de Evaluación

10.1. Distribución General

- **Código Funcional:** 40 % (40 puntos)
- **Defensa del Código:** 60 % (60 puntos)
- **Total:** 100 puntos
- **Puntos Extra:** Hasta +35 puntos adicionales

10.2. Parte 1: Código Funcional (40 puntos)

10.2.1. 1.1 Uso de Módulos Estándar (20 puntos)

- **Módulo random (5 pts):**
 - 5 pts: Usa 3+ funciones diferentes apropiadamente
 - 4 pts: Usa 2 funciones correctamente
 - 2 pts: Usa solo 1 función
 - 0 pts: No usa random o incorrectamente
- **Módulo statistics (5 pts):**
 - 5 pts: Usa 3+ funciones para análisis de datos
 - 4 pts: Usa 2 funciones correctamente
 - 2 pts: Uso mínimo
 - 0 pts: No usa statistics
- **Módulo datetime (4 pts):**
 - 4 pts: Usa para fechas, horas, formateo y cálculos
 - 3 pts: Uso correcto pero limitado
 - 1 pt: Uso mínimo
 - 0 pts: No usa datetime
- **Módulos os y sys (3 pts):**
 - 3 pts: Usa ambos para interacción con sistema
 - 2 pts: Usa uno correctamente
 - 1 pt: Uso mínimo
 - 0 pts: No usa ninguno
- **Importaciones correctas (3 pts):**
 - 3 pts: Todas las importaciones bien organizadas y apropiadas
 - 2 pts: Importaciones funcionales con errores menores
 - 1 pt: Importaciones desorganizadas
 - 0 pts: Importaciones incorrectas

10.2.2. 1.2 Módulo Personalizado (10 puntos)

- **Creación del módulo (4 pts):**
 - 4 pts: Archivo separado con funciones bien definidas
 - 3 pts: Módulo funcional con errores menores
 - 1 pt: Intento de módulo pero incompleto
 - 0 pts: No crea módulo propio
- **Funciones en el módulo (3 pts):**
 - 3 pts: 4+ funciones útiles y bien documentadas
 - 2 pts: 2-3 funciones
 - 1 pt: Solo 1 función
 - 0 pts: Sin funciones útiles
- **Uso del módulo (3 pts):**
 - 3 pts: Importa y usa correctamente en programa principal
 - 2 pts: Importa pero uso limitado
 - 1 pt: Importación con errores
 - 0 pts: No usa su propio módulo

10.2.3. 1.3 Funcionalidad del Programa (10 puntos)

- **Compra de billetes (3 pts):**
 - 3 pts: Sistema completo con validaciones
 - 2 pts: Funcional con errores menores
 - 1 pt: Implementación básica
 - 0 pts: No funciona
- **Sistema de sorteo (3 pts):**
 - 3 pts: Sorteo completo con todos los premios
 - 2 pts: Sorteo funcional parcialmente
 - 1 pt: Sorteo básico
 - 0 pts: No implementa sorteo
- **Estadísticas (2 pts):**
 - 2 pts: Estadísticas completas con statistics
 - 1 pt: Estadísticas básicas
 - 0 pts: No calcula estadísticas
- **Persistencia de datos (2 pts):**
 - 2 pts: Usa os para crear carpetas y guardar datos
 - 1 pt: Guarda algunos datos
 - 0 pts: No guarda datos

10.3. Parte 2: Defensa del Código (60 puntos)

10.3.1. 2.1 Comprensión de Módulos (30 puntos)

■ **Explicación de importaciones (8 pts):**

- 8 pts: Explica perfectamente import, from...import, diferencias
- 6 pts: Explica correctamente con pequeñas dudas
- 3 pts: Explicación básica
- 0 pts: No entiende importaciones

■ **Explicación de módulos usados (12 pts):**

- 12 pts: Explica claramente qué hace cada módulo y por qué lo usó
- 9 pts: Explica la mayoría correctamente
- 5 pts: Explicación superficial
- 0 pts: No entiende los módulos que usó

■ **Instalación de paquetes externos (6 pts):**

- 6 pts: Explica pip, requirements.txt, instalación correctamente
- 4 pts: Conoce el proceso con pequeñas dudas
- 2 pts: Conocimiento básico
- 0 pts: No sabe instalar paquetes

■ **Creación de módulos propios (4 pts):**

- 4 pts: Explica perfectamente cómo y por qué creó su módulo
- 3 pts: Explicación correcta
- 1 pt: Explicación vaga
- 0 pts: No entiende su propio módulo

10.3.2. 2.2 Demostración y Razonamiento (20 puntos)

■ **Demostración de funcionalidades (8 pts):**

- 8 pts: Demuestra todas las funcionalidades exitosamente
- 6 pts: Demuestra la mayoría
- 3 pts: Demostración parcial
- 0 pts: No puede demostrar funcionamiento

■ **Instalación de dependencias (4 pts):**

- 4 pts: Demuestra instalación con pip y requirements.txt
- 3 pts: Demuestra pero con errores menores
- 1 pt: Tiene dificultades
- 0 pts: No puede instalar dependencias

■ **Respuesta a preguntas técnicas (8 pts):**

- 8 pts: Responde correctamente todas las preguntas sobre módulos
- 6 pts: Responde la mayoría correctamente
- 3 pts: Respuestas parciales
- 0 pts: No puede responder preguntas básicas

10.3.3. 2.3 Calidad y Organización (10 puntos)

■ Estructura del proyecto (4 pts):

- 4 pts: Archivos bien organizados, estructura clara
- 3 pts: Organización aceptable
- 1 pt: Desorganizado
- 0 pts: Sin estructura

■ Documentación (3 pts):

- 3 pts: README, docstrings, comentarios claros
- 2 pts: Documentación básica
- 1 pt: Documentación mínima
- 0 pts: Sin documentación

■ Requirements.txt (3 pts):

- 3 pts: Archivo correcto con todas las dependencias
- 2 pts: Archivo presente con errores menores
- 1 pt: Incompleto
- 0 pts: No incluye requirements.txt

10.4. Puntos Extra (Hasta +35 puntos)

- Módulo JSON (+6 pts): Persistencia de datos en formato JSON
- Gráficos con matplotlib (+8 pts): Visualizaciones de estadísticas
- Módulo collections (+5 pts): Counter, defaultdict para análisis
- Módulo argparse (+5 pts): Argumentos de línea de comandos
- Módulo pickle (+4 pts): Serialización binaria de datos
- Crear paquete completo (+7 pts): Estructura con `__init__.py`

Nota: Los puntos extra solo se otorgan si la implementación es correcta Y el estudiante puede explicarla completamente durante la defensa.

10.5. Penalizaciones

- **-100 % de la defensa:** No asiste sin justificación (nota máxima = 40/100)
- **-50 % de la defensa:** Evidencia de no entender los módulos usados
- **-5 pts:** Entrega tardía de 1 día
- **-10 pts:** Entrega tardía de 2-3 días
- **-20 pts:** Entrega tardía de 4+ días
- **-5 pts:** No incluye módulo personalizado
- **-3 pts:** No incluye requirements.txt
- **-3 pts:** Usa `from modulo import *` (mala práctica)
- **-5 pts:** No usa al menos 3 módulos diferentes

10.6. Nota Mínima Aprobatoria

Para aprobar esta tarea necesitas obtener al menos **60 puntos de 100**.

10.7. Escala de Calificación

- **90-100+ pts:** Excelente - Dominio completo de módulos y librerías
- **80-89 pts:** Muy Bueno - Comprensión sólida del uso de librerías
- **70-79 pts:** Bueno - Comprensión aceptable, necesita práctica
- **60-69 pts:** Suficiente - Comprensión básica, necesita refuerzo
- **0-59 pts:** Insuficiente - No aprobado

11. Recursos de Apoyo

- **Documentación oficial:** <https://docs.python.org/es/3/tutorial/modules.html>
- **PyPI (paquetes):** <https://pypi.org/>
- **Módulo random:** <https://docs.python.org/es/3/library/random.html>
- **Módulo statistics:** <https://docs.python.org/es/3/library/statistics.html>
- **Módulo datetime:** <https://docs.python.org/es/3/library/datetime.html>
- **Guía de pip:** https://pip.pypa.io/en/stable/user_guide/

12. Preguntas Frecuentes

P: ¿Cuál es la diferencia entre `import random` y `from random import randint`?

R: Con `import random` usas `random.randint()`. Con `from random import randint` usas solo `randint()`.

P: ¿Qué es `pip`?

R: Es el gestor de paquetes de Python. Permite instalar librerías externas.

P: ¿Cómo instalo `colorama`?

R: Ejecuta en la terminal: `pip install colorama`

P: ¿Qué pasa si no tengo `pip`?

R: `Pip` viene incluido con Python 3.4+. Si no lo tienes, instala Python de nuevo.

P: ¿Puedo usar otros módulos no mencionados?

R: Sí! Mientras cumplas con los requeridos, puedes usar otros adicionales.

P: ¿Cómo creo un módulo propio?

R: Simplemente crea un archivo `.py` con funciones, luego importalo con `import nombre_archivo`

P: ¿Qué es `__name__ == "__main__"`?

R: Permite ejecutar código solo cuando el archivo se ejecuta directamente, no cuando se importa.

¡Éxito con tu simulador de lotería!

Los módulos y librerías son el superpoder de Python.

Aprende a usarlos y tendrás herramientas para cualquier problema.